ANIZA CORP

IamWatching v1.3.0

Multi-Cloud IAM Security Auditor

User & Configuration Manual

Complete step-by-step guide for all skill levels

Table of Contents

Table of Contents

Table of Contents 2
Chapter 1 — Introduction 4
1.1 What Is IamWatching? 4
1.2 What Does It Check? 4
1.3 Key Features 4
Chapter 2 — Installation 6
2.1 What You Need Before Starting 6
2.2 Step-by-Step Installation 6
Step 1 — Download the Package 6
Step 2 — Open a Terminal 6
Step 3 — Create a Fresh Project Folder 6
Step 4 — Run the Deployment Script 7
Step 5 — Start the Services 7
Step 6 — Activate the Virtual Environment 8
2.3 Verifying the Installation 8
Chapter 3 — Setting Up Cloud Credentials 9
3.1 AWS Credentials 9
Option A — AWS CLI Profile (Recommended for Most Users) 9
Option B — Environment Variables 9
Minimum AWS Permissions Required 9
3.2 Azure Credentials 10
Option A — Azure CLI (Recommended) 10
Option B — Service Principal (for Automated/CI Environments) 10
3.3 GCP Credentials 10
Option A — gcloud CLI (Recommended) 10
Option B — Service Account Key File 11
Chapter 4 — Running Security Audits 12
4.1 Your First Audit 12
4.2 Common Audit Commands 12
4.3 Understanding the Results 12
4.4 Severity Levels Explained 13
Chapter 5 — Security Checks 14
5.1 Listing Available Checks 14
5.2 The Checks Command 14

5.3 Adding Your Own Custom Checks **15**
Method 1 — Using the Command Line (Easiest) 15
Method 2 — Writing a YAML File (Most Flexible) 15
5.4 Check YAML Reference **16**
5.5 Writing Cypher Queries for Custom Checks **17**
Pattern 1 — Find Resources Missing Something 17
Pattern 2 — Find Resources With a Dangerous Property 17
Pattern 3 — Find Trust Relationships 17
5.6 Updating Built-In Checks **18**
5.7 Disabling Specific Checks **18**
Chapter 6 — Exploring Results in Neo4j ***20***
6.1 What Is Neo4j? **20**
6.2 Useful Starter Queries **20**
See Everything 20
List All IAM Users 20
Find Admin-Level Roles 20
Find Cross-Cloud Credential Leaks 20
Show Trust Relationships 21
Chapter 7 — Configuration Reference ***22***
7.1 Deployment Options **22**
7.2 Audit Command Options **22**
7.3 Environment Variables **23**
Chapter 8 — Troubleshooting ***24***
8.1 Common Errors and Fixes **24**
Error: No module named 'aioboto3' 24
Error: AWS auth failed — No credentials 24
Error: Neo4j not reachable at bolt://localhost:7687 24
Error: permission denied: ./iamwatching 24
Deployment stops at 'Running pytest' 24
Error: iamwatching/iamwatching/iamwatching/... in log paths 25
8.2 Getting Help **25**
Chapter 9 — Quick Reference Card ***26***
Daily Workflow **26**
Check Management Quick Reference **26**
Family Filter Values **26**
Checks Directory Structure **26**
Appendix — Complete Check Library ***28***

Chapter 1 — Introduction

1.1 What Is IamWatching?

IamWatching is a security tool that automatically checks whether the people and services in your cloud accounts (Amazon Web Services, Microsoft Azure, and Google Cloud Platform) have the right level of access — not too much, not too little.

Think of it like a security guard that walks through your cloud and asks: "Does this person really need access to everything?" and "Is anyone hiding credentials where they shouldn't be?"

> ℹ️ **INFO**
>
> You do not need to be a programmer to use IamWatching. This manual will guide you through every step, explaining what each action does and why.

1.2 What Does It Check?

IamWatching runs security checks from three internationally recognised security frameworks:

Framework	What It Is	Checks Included
CIS AWS Foundations v3.0	Center for Internet Security — best practices for AWS	9 checks
OWASP Cloud-Native Top 10	Open Web Application Security Project — cloud risks	6 checks
NIST CSF 2.0	US National Institute of Standards — identity controls	5 checks
Custom (yours)	Checks you write yourself for your organisation	Unlimited

It also scans your cloud resources (Lambda functions, virtual machines, containers) looking for hidden credentials — for example, an AWS key accidentally saved inside a Google Cloud function.

1.3 Key Features

- **Multi-cloud:** scans AWS, Azure, and GCP in a single command
- **Live graph database:** stores all relationships in Neo4j so you can explore interactively
- **Custom checks:** write your own YAML rules without coding knowledge
- **Severity ratings:** every finding is rated CRITICAL / HIGH / MEDIUM / LOW / INFO

- **MITRE ATT&CK mapped:** findings reference real attacker techniques
- **Docker-ready:** runs in containers with a single command

Chapter 2 — Installation

2.1 What You Need Before Starting

Before installing IamWatching, make sure you have the following installed on your computer. If you are not sure, ask your IT department.

Requirement	Minimum Version	How to Check	Download
Python	3.11 or newer	`python3 --version`	python.org
Docker Desktop	Latest	`docker --version`	docker.com
Git	Any recent	`git --version`	git-scm.com
AWS CLI	v2	`aws --version`	aws.amazon.com/cli

⚠ IMPORTANT

IamWatching requires Python 3.11 or newer. It will NOT work on older Python versions. Run python3 --version in your terminal to check.

2.2 Step-by-Step Installation

Step 1 — Download the Package

Download the file iamwatching-v1.3.0.tar.gz to your computer. Save it somewhere easy to find, such as your Downloads folder.

Step 2 — Open a Terminal

On Mac: press Command + Space, type "Terminal", press Enter.

On Windows: press the Windows key, type "Command Prompt" or "PowerShell", press Enter.

On Linux: press Ctrl + Alt + T.

Step 3 — Create a Fresh Project Folder

Type the following commands one at a time, pressing Enter after each:

```
# Create a folder for IamWatching
mkdir ~/IamWatching

# Move into that folder
cd ~/IamWatching

# Extract the downloaded package into this folder
```

```
tar xzf ~/Downloads/iamwatching-v1.3.0.tar.gz

# Move into the extracted folder
cd iamwatching
```

⚠ IMPORTANT

Important: Always extract into a NEW empty folder. If you extract multiple times into the same place, files will overlap and the tool will not work correctly.

Step 4 — Run the Deployment Script

The deployment script installs everything automatically. Run:

```
# Make the script executable and run it
chmod +x deploy.sh
./deploy.sh
```

The script will:

1. Check that Python 3.11+ is installed
2. Create a virtual environment (an isolated Python installation just for this tool)
3. Install all required packages (this may take 2-5 minutes)
4. Run automated tests to verify everything works
5. Build the Go background daemon
6. Build Docker containers
7. Create start.sh, stop.sh, and uninstall.sh scripts

💡 TIP

If any step fails, you will see a clear error message explaining what went wrong. The most common issue is a missing dependency — the error will tell you exactly what to install.

Step 5 — Start the Services

After deployment completes successfully:

```
./start.sh
```

This starts Neo4j (the graph database) and the background daemon. You will see:

```
══ Ready ══
  Venv audit    : iamwatching audit --aws     (activate .venv first)
  Native audit  : ./dist/iamwatching audit --aws   (no venv needed)
  Stop all      : ./stop.sh
```

Step 6 — Activate the Virtual Environment

Before using the iamwatching command, activate the virtual environment:

```
# On Mac/Linux:
source .venv/bin/activate

# You will see (.venv) appear at the start of your prompt
# Now you can run: iamwatching --help
```

TIP

Alternative — no activation needed: Use ./dist/iamwatching instead of iamwatching. This is the compiled binary that works without activating the virtual environment.

2.3 Verifying the Installation

Run the following to confirm everything installed correctly:

```
iamwatching --version
# Should output: iamwatching 1.3.0

iamwatching --help
# Should show a list of available commands

iamwatching audit --aws --list-checks
# Should show all 22+ security checks without connecting to AWS
```

Chapter 3 — Setting Up Cloud Credentials

IamWatching needs permission to read your cloud accounts. It never makes changes — it only reads. You need to give it "read-only" access before running any scans.

3.1 AWS Credentials

Option A — AWS CLI Profile (Recommended for Most Users)

If you already use the AWS CLI, IamWatching will automatically use your existing credentials. Run a scan like this:

```
iamwatching audit --aws --aws-profile your-profile-name --aws-regions us-
east-1
```

To see what profiles you have configured:

```
aws configure list-profiles
```

Option B — Environment Variables

Set these variables in your terminal before running a scan:

```
export AWS_ACCESS_KEY_ID=AKIA...your-key-here...
export AWS_SECRET_ACCESS_KEY=your-secret-key-here
export AWS_DEFAULT_REGION=us-east-1

# Then run your scan
iamwatching audit --aws --aws-regions us-east-1
```

NOTE

Never share your AWS access keys with anyone. Never commit them to Git. If you accidentally expose a key, rotate it immediately in the AWS console.

Minimum AWS Permissions Required

The IAM user or role you use must have at least these permissions (your IT administrator can create this):

```
# Permissions needed (read-only):
iam:GetAccountPasswordPolicy
iam:ListUsers
iam:ListRoles
iam:ListGroups
iam:ListPolicies
iam:GetPolicy
```

```
iam:GetPolicyVersion
iam:ListAttachedUserPolicies
iam:ListAttachedRolePolicies
iam:ListUserPolicies
iam:ListRolePolicies
s3:ListAllMyBuckets
s3:GetBucketPolicy
lambda:ListFunctions
lambda:GetFunction
ec2:DescribeInstances
sts:GetCallerIdentity
```

TIP

The easiest approach is to attach the AWS managed policy "ReadOnlyAccess" to your IAM user. This grants all the permissions IamWatching needs and nothing more.

3.2 Azure Credentials

Option A — Azure CLI (Recommended)

```
# Log in to Azure
az login

# Set your subscription
az account set --subscription YOUR_SUBSCRIPTION_ID

# Find your tenant ID
az account show --query tenantId

# Run the scan
iamwatching audit --azure \
  --azure-subscription YOUR_SUBSCRIPTION_ID \
  --azure-tenant YOUR_TENANT_ID
```

Option B — Service Principal (for Automated/CI Environments)

```
export AZURE_CLIENT_ID=your-app-id
export AZURE_CLIENT_SECRET=your-client-secret
export AZURE_TENANT_ID=your-tenant-id

iamwatching audit --azure \
  --azure-subscription YOUR_SUBSCRIPTION_ID \
  --azure-tenant YOUR_TENANT_ID
```

3.3 GCP Credentials

Option A — gcloud CLI (Recommended)

```
# Log in
gcloud auth application-default login
```

```
# Run the scan
iamwatching audit --gcp --gcp-project your-project-id
```

Option B — Service Account Key File

```
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account-key.json

iamwatching audit --gcp --gcp-project your-project-id
```

⚠ IMPORTANT

Never store service account key files in Git repositories or shared drives. Keep them in a secure location and restrict who can access them.

Chapter 4 — Running Security Audits

4.1 Your First Audit

The simplest possible audit — scan your AWS account in one region:

```
iamwatching audit --aws --aws-regions us-east-1
```

What happens when you run this:

8. IamWatching connects to AWS using your credentials
9. It reads all IAM users, roles, groups, and policies
10. It scans Lambda functions and EC2 instances for hidden credentials
11. It stores everything in the Neo4j graph database
12. It runs all 22+ security checks and reports findings
13. Results are shown on screen with severity ratings

4.2 Common Audit Commands

What You Want to Do	Command
Scan AWS, one region	`iamwatching audit --aws --aws-regions us-east-1`
Scan AWS, multiple regions	`iamwatching audit --aws --aws-regions us-east-1,eu-west-1`
Scan AWS with a named profile	`iamwatching audit --aws --aws-profile myprofile`
Scan Azure	`iamwatching audit --azure --azure-subscription SUB_ID --azure-tenant TENANT_ID`
Scan GCP	`iamwatching audit --gcp --gcp-project my-project`
Scan all three clouds	`iamwatching audit --aws --azure --gcp`
Only show CRITICAL findings	`iamwatching audit --aws --severity CRITICAL`
Save results to a file	`iamwatching audit --aws --output report.json`
Skip graph database	`iamwatching audit --aws --no-import-graph --no-detect`
Verbose output (for troubleshooting)	`iamwatching audit --aws --verbose`

4.3 Understanding the Results

After each scan, IamWatching shows a summary like this:

```
╭──────────────────────────── Audit Complete ────────────────────────────╮
│ Clouds scanned:                aws                                     │
│ Principals found:              47 (31 users, 16 roles)                 │
│ Resources scanned:             23                                      │
│ Cross-cloud credentials:       2 (VERIFIED LIVE)                       │
│ Security findings:             8                                       │
│   CRITICAL: 2  HIGH: 3  MEDIUM: 2  LOW: 1                              │
╰────────────────────────────────────────────────────────────────────────╯
```

Each finding includes:

- **Severity:** how serious the issue is (CRITICAL, HIGH, MEDIUM, LOW, INFO)
- **Title:** a plain-English name for the problem
- **Affected resources:** exactly which users, roles, or services are affected
- **Recommendation:** what to do to fix it
- **MITRE ATT&CK reference:** the real-world attack technique this finding corresponds to

4.4 Severity Levels Explained

Severity	Meaning	Action Required
CRITICAL	Immediate risk — attacker could gain full account control	Fix within 24 hours
HIGH	Significant risk — likely path to privilege escalation	Fix within 1 week
MEDIUM	Moderate risk — violates security best practices	Fix within 1 month
LOW	Minor issue — good hygiene improvement	Fix in next sprint
INFO	Informational — worth knowing, no immediate action needed	Review quarterly

Chapter 5 — Security Checks

5.1 Listing Available Checks

Before running a scan, you can see exactly which security checks will be performed. This never connects to any cloud account — it just shows you the list.

```
# Show ALL checks
iamwatching audit --aws --list-checks

# Show only OWASP checks
iamwatching audit --aws --list-checks --family owasp

# Show only CIS checks
iamwatching audit --aws --list-checks --family cis

# Show only NIST checks
iamwatching audit --aws --list-checks --family nist

# Show your custom checks
iamwatching audit --aws --list-checks --family custom

# Combine multiple families
iamwatching audit --aws --list-checks --family cis --family owasp
```

The output shows each check with its ID, severity, title, and whether it is currently enabled (✓) or disabled (✗).

5.2 The Checks Command

Use the iamwatching checks command to manage your security checks:

Command	What It Does
`iamwatching checks list`	Show all checks in a table
`iamwatching checks list --framework CIS-AWS-3.0`	Show only CIS checks
`iamwatching checks show CIS-AWS-1.4`	Show full details of one check
`iamwatching checks disable CIS-AWS-1.17`	Turn off a check (it will be skipped)
`iamwatching checks enable CIS-AWS-1.17`	Turn a check back on
`iamwatching checks reload`	Pick up new YAML files without restarting
`iamwatching checks add --id ... --title ...`	Add a new custom check

Command	What It Does
`iamwatching checks update`	Show instructions for updating built-in checks

5.3 Adding Your Own Custom Checks

You can write your own security checks in YAML format — no programming knowledge required. Custom checks live in the checks/custom/ folder and are never overwritten when you update IamWatching.

Method 1 — Using the Command Line (Easiest)

For simple checks, use the add command:

```
iamwatching checks add \
  --id CUSTOM-001 \
  --title "All production roles must have a description" \
  --severity MEDIUM \
  --framework CUSTOM \
  --description "Production IAM roles without descriptions are hard to audit"
\
  --recommendation "Add a description to every IAM role explaining its
purpose" \
  --cypher "MATCH (r:AWSPrincipal {principal_type: 'Role'}) WHERE NOT
r.metadata CONTAINS 'description' RETURN r.arn AS role"
```

The check is immediately saved to checks/custom/ and takes effect next time you run checks reload.

Method 2 — Writing a YAML File (Most Flexible)

Create a new file inside the checks/custom/ folder. Name it anything you like, ending in .yaml. Here is a complete example:

```
# checks/custom/my_company_checks.yaml

framework: CUSTOM
description: My company security requirements

checks:

  - id: CUSTOM-010
    title: Service accounts must follow naming convention svc-*
    severity: LOW
    description: >
      All service accounts must start with 'svc-' so they can be
      easily identified in audit logs and distinguished from human users.
    cypher: |
      MATCH (p:AWSPrincipal {principal_type: 'User'})
      WHERE size(p.attached_policies) > 0
```

```
        AND NOT p.name STARTS WITH 'svc-'
      RETURN p.arn AS user_arn, p.name AS username
    recommendation: >
      Rename service accounts to follow the svc-<service>-<env> convention.
      Example: svc-payments-prod, svc-reporting-staging
    mitre:
      - T1078.004 - Cloud Accounts
    references:
      - https://your-internal-wiki.com/iam-naming

  - id: CUSTOM-011
    title: All resources must have an 'owner' tag
    severity: MEDIUM
    description: >
      Resources without an owner tag cannot be attributed to a team
      during a security incident. This slows down response time.
    cypher: |
      MATCH (r:Resource)
      WHERE NOT r.tags CONTAINS '"owner"'
      RETURN labels(r) AS cloud, r.name AS name,
             COALESCE(r.arn, r.resource_id) AS resource_id
      ORDER BY cloud, name
      LIMIT 100
    recommendation: >
      Add an 'owner' tag to all resources using your infrastructure-as-code
tool.
      Enforce this via AWS Organizations Tag Policies or equivalent.
    mitre: []
    references: []
```

After saving the file, reload the checks:

```
iamwatching checks reload
# Output: Reloaded 24 checks.
#   CIS-AWS-3.0: 9
#   CUSTOM: 4      <-- your new checks appear here
#   NIST-CSF-2.0: 5
#   OWASP-CLOUD-NATIVE-2024: 6
```

TIP

Custom checks with the same ID as a built-in check will replace that built-in check. This lets you modify the existing CIS or OWASP checks to fit your specific environment.

5.4 Check YAML Reference

Here is every field you can use in a check YAML file:

Field	Required?	Description
id	Yes	Unique identifier e.g. CUSTOM-001. Must be unique across all checks.

Field	Required?	Description
title	Yes	Short, plain-English name shown in reports (under 80 characters)
severity	Yes	One of: CRITICAL, HIGH, MEDIUM, LOW, INFO
cypher	Yes	The Neo4j graph query that finds affected resources (see section 5.5)
description	No	Longer explanation of the security risk (shown in checks show)
recommendation	No	How to fix the finding — step-by-step if possible
framework	No	Framework name (defaults to CUSTOM if not set)
mitre	No	List of MITRE ATT&CK technique IDs e.g. T1078.004
references	No	List of URLs for further reading
enabled	No	true or false — set to false to disable a check (default: true)

5.5 Writing Cypher Queries for Custom Checks

Cypher is the query language for the Neo4j graph database. You do not need to be an expert — most custom checks follow one of three simple patterns:

Pattern 1 — Find Resources Missing Something

```
# Find all AWS roles that have no description
MATCH (r:AWSPrincipal {principal_type: 'Role'})
WHERE NOT r.metadata CONTAINS '"description"'
RETURN r.arn AS role_arn, r.name AS role_name
```

Pattern 2 — Find Resources With a Dangerous Property

```
# Find resources with discovered cross-cloud credentials
MATCH (r:Resource)-[e:CROSS_CLOUD_LINK]->(tgt)
WHERE e.verified = true
RETURN r.name AS resource, e.cred_type AS credential_type,
       e.target_cloud AS leaks_to_cloud
```

Pattern 3 — Find Trust Relationships

```
# Find roles that can be assumed from external accounts
MATCH (src:AWSPrincipal)-[e:CAN_ASSUME]->(tgt:AWSPrincipal)
WHERE src.account_id <> tgt.account_id
RETURN src.arn AS external_caller,
       tgt.arn AS assumed_role,
       e.condition AS trust_condition
```

Graph Label	What It Represents
AWSPrincipal	An IAM user, role, or group in AWS
AzurePrincipal	A user, service principal, or managed identity in Azure
GCPPrincipal	A service account or user in GCP
AWSResource	An S3 bucket, Lambda function, EC2 instance, or ECS task in AWS
AzureResource	A Function App or Virtual Machine in Azure
GCPResource	A Cloud Function, Cloud Run service, or Compute instance in GCP
Principal	Any identity across any cloud
Resource	Any resource across any cloud

TIP

To explore your data interactively and test Cypher queries, open the Neo4j Browser at http://localhost:7474 (user: neo4j, password: iamwatching by default). Try: MATCH (n) RETURN n LIMIT 25 to see your graph.

5.6 Updating Built-In Checks

When new versions of CIS, OWASP, or NIST standards are released, you can update the built-in check files manually:

14. Download the updated YAML files from the IamWatching release page
15. **Replace** the files in checks/builtin/ (e.g. cis_aws.yaml)
16. Run: iamwatching checks reload

Your custom checks in checks/custom/ are never touched during this process.

```
# Check what version of each framework you have
iamwatching checks list --framework CIS-AWS-3.0

# After updating files:
iamwatching checks reload

# Show update instructions
iamwatching checks update
```

5.7 Disabling Specific Checks

Some checks may not apply to your organisation. You can disable them so they do not appear in reports:

```
# Disable a check
iamwatching checks disable CIS-AWS-1.17
# Output: Disabled: CIS-AWS-1.17

# Re-enable it later
iamwatching checks enable CIS-AWS-1.17
# Output: Enabled: CIS-AWS-1.17

# Run with only specific families
iamwatching audit --aws --family cis
```

TIP

Disabling a check is temporary — it resets when you restart IamWatching. To permanently disable a check, add enabled: false to the YAML file.

Chapter 6 — Exploring Results in Neo4j

6.1 What Is Neo4j?

Neo4j is a graph database — it stores data as a network of connected nodes (dots) and relationships (lines). It is perfect for IAM because IAM is all about who is connected to what: user A has role B which has permission C on resource D.

After a scan, open the Neo4j Browser in your web browser:

```
http://localhost:7474

Username: neo4j
Password: iamwatching   (or whatever you set with --neo4j-password during
deploy)
```

6.2 Useful Starter Queries

Type these in the Neo4j Browser query box and press Ctrl+Enter (or click the play button):

See Everything

```
# Show a sample of all nodes
MATCH (n) RETURN n LIMIT 50
```

List All IAM Users

```
MATCH (u:AWSPrincipal {principal_type: 'User'})
RETURN u.name AS username, u.arn AS arn
ORDER BY username
```

Find Admin-Level Roles

```
MATCH (p:Principal)-[e:HAS_PERMISSION]->(r)
WHERE e.action IN ['*', 'iam:*', 'AdministratorAccess']
RETURN p.name AS principal, e.action AS permission
```

Find Cross-Cloud Credential Leaks

```
MATCH (src:Resource)-[e:CROSS_CLOUD_LINK]->(tgt)
RETURN src.name AS leaky_resource,
       e.cred_type AS credential_type,
       e.target_cloud AS leaks_to,
       e.verified AS confirmed_live
ORDER BY confirmed_live DESC
```

Show Trust Relationships

```
MATCH (a:AWSPrincipal)-[r:CAN_ASSUME]->(b:AWSPrincipal)
RETURN a.name AS from_principal, b.name AS to_role
LIMIT 25
```

Chapter 7 — Configuration Reference

7.1 Deployment Options

Pass these options when running ./deploy.sh to customise the installation:

Option	Default	What It Does
`--skip-tests`	off	Skip automated tests (faster deployment, not recommended)
`--skip-docker`	off	Do not build Docker containers
`--skip-daemon`	off	Do not build the Go background daemon
`--native`	off	Also build a self-contained native binary (takes 2-3 minutes extra)
`--dev`	off	Install development tools (for contributors)
`--log-level DEBUG`	INFO	Show detailed deployment output
`--log-dir /path`	./logs	Where to write log files
`--neo4j-password secret`	iamwatching	Set a custom Neo4j password

```
# Example: deploy without tests, custom password, extra verbose
./deploy.sh --skip-tests --neo4j-password MySecurePassword123 --log-level
DEBUG
```

7.2 Audit Command Options

Option	Example	What It Does
`--aws`	`--aws`	Enable AWS scanning
`--azure`	`--azure`	Enable Azure scanning
`--gcp`	`--gcp`	Enable GCP scanning
`--aws-profile`	`--aws-profile prod`	Use a named AWS CLI profile
`--aws-regions`	`--aws-regions us-east-1,eu-west-1`	AWS regions to scan (comma-separated)
`--azure-subscription`	`--azure-subscription abc-123`	Azure subscription ID
`--azure-tenant`	`--azure-tenant abc-123`	Azure tenant (directory) ID
`--gcp-project`	`--gcp-project my-project`	GCP project ID
`--verify`	`--verify`	Test if discovered credentials are still valid (default: on)

Option	Example	What It Does
`--no-import-graph`	`--no-import-graph`	Skip writing to Neo4j
`--no-detect`	`--no-detect`	Skip running pattern detection
`--severity`	`--severity HIGH`	Show only findings at this level or above
`--output`	`--output report.json`	Save full results to a JSON file
`--list-checks`	`--list-checks`	Show checks without scanning; then exit
`--family`	`--family owasp`	Filter checks by framework family
`--verbose`	`--verbose`	Enable detailed debug output

7.3 Environment Variables

These environment variables can be set instead of passing command-line flags:

Variable	What It Controls
AWS_ACCESS_KEY_ID	AWS access key (alternative to --aws-profile)
AWS_SECRET_ACCESS_KEY	AWS secret key
AWS_DEFAULT_REGION	Default AWS region
AWS_PROFILE	AWS CLI profile name
AZURE_CLIENT_ID	Azure service principal application ID
AZURE_CLIENT_SECRET	Azure service principal secret
AZURE_TENANT_ID	Azure directory (tenant) ID
GOOGLE_APPLICATION_CREDENTIALS	Path to GCP service account key JSON file
NEO4J_URI	Neo4j connection string (default: bolt://localhost:7687)
NEO4J_USERNAME	Neo4j username (default: neo4j)
NEO4J_PASSWORD	Neo4j password (default: iamwatching)
IAMWATCHING_CHECKS_DIR	Override the checks/ directory location

Chapter 8 — Troubleshooting

8.1 Common Errors and Fixes

Error: No module named 'aioboto3'

```
# The cloud SDKs are not installed. Fix:
pip install -e '.[dev]'
# or:
pip install aioboto3 aiobotocore botocore
```

Error: AWS auth failed — No credentials

```
# Check your credentials are set:
aws sts get-caller-identity

# If that fails, configure AWS CLI:
aws configure

# Or set environment variables:
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
```

Error: Neo4j not reachable at bolt://localhost:7687

```
# Neo4j is not running. Start it:
./start.sh

# Or start just Neo4j via Docker:
docker compose -f docker/docker-compose.yml up -d neo4j

# Check it is running:
docker ps | grep neo4j

# To scan without Neo4j (no graph storage):
iamwatching audit --aws --no-import-graph --no-detect
```

Error: permission denied: ./iamwatching

```
# You are trying to run the directory, not the binary.
# The binary is in dist/. Use:
./dist/iamwatching audit --aws

# Or activate the virtual environment first:
source .venv/bin/activate
iamwatching audit --aws
```

Deployment stops at 'Running pytest'

```
# Skip tests and deploy anyway:
./deploy.sh --skip-tests

# Or run tests manually to see what failed:
source .venv/bin/activate
python3 -m pytest tests/ -v --tb=short
```

Error: iamwatching/iamwatching/iamwatching/... in log paths

You have extracted the tarball into a folder that already contains a previous extraction. Fix:

```
# Create a completely new empty folder:
mkdir ~/IamWatchingFresh
cd ~/IamWatchingFresh
tar xzf ~/Downloads/iamwatching-v1.3.0.tar.gz
cd iamwatching
./deploy.sh
```

8.2 Getting Help

For help with any command, add --help:

```
iamwatching --help
iamwatching audit --help
iamwatching checks --help
iamwatching checks add --help
```

To see what version you are running:

```
iamwatching --version
```

Chapter 9 — Quick Reference Card

Daily Workflow

```
# 1. Start services
./start.sh

# 2. Activate virtual environment
source .venv/bin/activate

# 3. Run a scan
iamwatching audit --aws --aws-regions us-east-1

# 4. Browse results in Neo4j
open http://localhost:7474

# 5. When done
./stop.sh
```

Check Management Quick Reference

```
iamwatching audit --aws --list-checks              # See all checks
iamwatching audit --aws --list-checks --family cis # CIS checks only
iamwatching checks list                            # Detailed check list
iamwatching checks show CIS-AWS-1.4               # Full check details
iamwatching checks disable CIS-AWS-1.17            # Disable a check
iamwatching checks enable CIS-AWS-1.17             # Re-enable a check
iamwatching checks reload                          # Load new YAML files
iamwatching checks add --id X --title Y ...        # Add a check via CLI
# Or: create checks/custom/my_checks.yaml manually
```

Family Filter Values

--family value	Framework matched
`owasp`	OWASP Cloud-Native Application Security Top 10
`cis`	CIS Amazon Web Services Foundations Benchmark v3.0
`nist`	NIST Cybersecurity Framework 2.0
`custom`	Your custom checks in checks/custom/

Checks Directory Structure

```
iamwatching/
├── checks/
│   ├── builtin/              ← Built-in checks (do not edit these directly)
│   │   ├── cis_aws.yaml      ← 9 CIS AWS checks
│   │   ├── owasp_cloud.yaml  ← 6 OWASP checks
│   │   ├── nist_csf.yaml     ← 5 NIST CSF checks
```

```
│   │   └── custom_example.yaml← Example custom check template
│   └── custom/                ← YOUR checks go here ← never overwritten
│       └── my_checks.yaml     ← Create any .yaml files here
├── deploy.sh                  ← Run this first to install
├── start.sh                   ← Start Neo4j + daemon
├── stop.sh                    ← Stop everything
└── uninstall.sh               ← Remove all components
```

Appendix — Complete Check Library

The following table lists all 22 built-in security checks included in IamWatching v1.3.0:

Check ID	Severity	Framework	Title
CIS-AWS-1.4	CRITICAL	CIS AWS 3.0	Root account access key should not exist
CIS-AWS-1.5	CRITICAL	CIS AWS 3.0	MFA must be enabled for root account
CIS-AWS-1.6	HIGH	CIS AWS 3.0	MFA should be enabled for all IAM users
CIS-AWS-1.7	MEDIUM	CIS AWS 3.0	Password policy should require 14+ characters
CIS-AWS-1.9	HIGH	CIS AWS 3.0	Access keys must be rotated every 90 days
CIS-AWS-1.10	MEDIUM	CIS AWS 3.0	Users should not have both console and key access
CIS-AWS-1.16	MEDIUM	CIS AWS 3.0	IAM policies must not attach directly to users
CIS-AWS-1.17	LOW	CIS AWS 3.0	Maintain current AWS account contact information
CIS-AWS-1.20	LOW	CIS AWS 3.0	Create a support role for incident management
OWASP-CNAS-1	HIGH	OWASP 2024	Insecure cloud / container configuration
OWASP-CNAS-2	HIGH	OWASP 2024	Broken authentication (overprivileged service identities)
OWASP-CNAS-3	HIGH	OWASP 2024	Insecure third-party dependencies (external trust)
OWASP-CNAS-4	MEDIUM	OWASP 2024	Improper asset and permission scope
OWASP-CNAS-6	CRITICAL	OWASP 2024	Secrets and credentials in compute environment variables
OWASP-CNAS-8	HIGH	OWASP 2024	Container escape via overprivileged task identity
NIST-PR.AA-01	MEDIUM	NIST CSF 2.0	Identities and credentials are managed
NIST-PR.AA-02	HIGH	NIST CSF 2.0	Identities bound to specific workloads
NIST-PR.AA-05	HIGH	NIST CSF 2.0	Access permissions and authorizations managed
NIST-PR.AA-06	LOW	NIST CSF 2.0	Physical and logical assets inventoried (tags)
NIST-DE.CM-03	MEDIUM	NIST CSF 2.0	Personnel activity and technology usage monitored
CUSTOM-001	MEDIUM	CUSTOM	Example: Resources must have required tags

Check ID	Severity	Framework	Title
CUSTOM-002	LOW	CUSTOM	Example: Service accounts must follow svc-* naming